

Proves

R4 Cheng

February 6, 2026

Math

Quadratic Form

$$\nabla \vec{x}^T \mathbf{A} \vec{x} = \vec{x}^T (\mathbf{A} + \mathbf{A}^T)$$

if $\mathbf{A}$ is symmetric, then $\nabla_{\vec{x}} \vec{x}^T \mathbf{A} \vec{x} = 2\vec{x}^T \mathbf{A}$

Why is this important?

Almost all smooth nonlinear functions can be approximated by a quadratic function near a local minimum or maximum.

Inverse Matrix

$$[A \mid I] \rightarrow [I \mid A^{-1}]$$

E.g. $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

$$\begin{aligned} & \left[\begin{array}{cc|cc} 1 & 2 & 1 & 0 \\ 3 & 4 & 0 & 1 \end{array} \right] \\ \Rightarrow & \left[\begin{array}{cc|cc} 1 & 0 & -2 & 1 \\ 0 & 1 & 1.5 & -0.5 \end{array} \right] \\ \Rightarrow & A^{-1} = \begin{bmatrix} -2 & 1 \\ 1.5 & -0.5 \end{bmatrix} \end{aligned}$$

Symmetric Matrix

$$\mathbf{A} = \mathbf{A}^T$$

Hessian Matrix

- It is a symmetric matrix.

$$H = \nabla^2 f(\vec{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{bmatrix} = \begin{bmatrix} f_{xx} & f_{xy} \\ f_{yx} & f_{yy} \end{bmatrix}$$

> For continuous functions, $f_{xy} = f_{yx}$, so Hessian matrix is symmetric.

Determinant of Hessian:

$$H = \begin{bmatrix} A & B \\ B & C \end{bmatrix} \Rightarrow \det(H) = AC - B^2$$

Find Eigenvalues of Hessian

Build characteristic equation:

$$\begin{aligned} \det(H - \lambda I) &= 0 \\ \Rightarrow \det \begin{bmatrix} f_{xx} - \lambda & f_{xy} \\ f_{yx} & f_{yy} - \lambda \end{bmatrix} &= 0 \\ \Rightarrow (f_{xx} - \lambda)(f_{yy} - \lambda) - (f_{xy})(f_{yx}) &= 0, (a\lambda^2 + b\lambda + c = 0) \end{aligned}$$

The roots λ_1 and λ_2 are the eigenvalues of H .

For each critical point x_0 :

- If both eigenvalues are positive, then x_0 is a local minimum.
- If both eigenvalues are negative, then x_0 is a local maximum.
- If eigenvalues have different signs, then x_0 is a saddle point.
- If any eigenvalue is zero, the test is inconclusive.

Jacobian Matrix

$$J = \frac{\partial \sigma(\mathbf{x})}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial \sigma(x_1)}{\partial x_1} & \dots & \frac{\partial \sigma(x_1)}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial \sigma(x_n)}{\partial x_1} & \dots & \frac{\partial \sigma(x_n)}{\partial x_n} \end{bmatrix}$$

Since Sigmoid function is element-wise, the output $\sigma(x_i)$ only depends on input x_i (other inputs are irrelevant). Therefore, all non-diagonal entries of the Jacobian matrix are zero.

$$\Rightarrow J = \text{diag}(\sigma(\mathbf{x}) \odot (\mathbf{1} - \sigma(\mathbf{x})))$$

$$(\text{Streamline}) \sigma'(\vec{x}) = \sigma(\vec{x}) \odot (\vec{1} - \sigma(\vec{x}))$$

> A small linear algebra trick: multiplying a row vector by a diagonal matrix is equivalent to performing element-wise multiplication (the Hadamard product) between the two vectors

Loss (Cost) function

> aka. how to reward a machine

L_2 :

$$J(\vec{w}) = \frac{1}{2} \sum_{(\vec{x}, y) \in D} (f_{\vec{w}}(\vec{x}) - y)^2 + \lambda \|\vec{w}\|_2^2$$

> Euclidean norm: $\|\vec{w}\|_2 = \sqrt{w_0^2 + w_1^2 + \dots + w_n^2}$

L_1 :

$$J(\vec{w}) = \frac{1}{2} \sum_{(\vec{x}, y) \in D} (f_{\vec{w}}(\vec{x}) - y)^2 + \lambda \|\vec{w}\|_1^2$$

> Manhattan norm: $\|\vec{w}\|_1 = |w_0| + |w_1| + \dots + |w_n|$

Target:

$$\min_{\vec{w}} J(\vec{w})$$

Loss functions for classification

$$\min_{\theta} \mathbb{E}_{x,y} [\mathbb{I}[yf(x; \theta) \leq 0]]$$

If we are relying on gradients; why we pick a loss function that is upper bound of the 0-1 loss?

$$L_{Proxy}(x) \geq L_{0/1}(x)$$

A: As long as you can drive the upper bound very low, your true error rate will definitely also be low.

Options:

- L1: $L_1 = |yf(x; \theta) - 1|$
- L2: $L_2 = (yf(x; \theta) - 1)^2$
- Hinge: $L_{\text{Hinge}} = \max(0, 1 - yf(x; \theta))$
- Cross Entropy: $L_{\text{CE}} = \log_2(1 + e^{-yf(x; \theta)})$

Loss functions for Regression

- L2 Loss (Squared Error / MSE): $L_2(y, f(x)) = (y - f(x))^2$
- L1 Loss (Absolute Error / MAE): $L_1(y, f(x)) = |y - f(x)|$
- Huber Loss:

$$L_\delta(y, f(x)) = \begin{cases} \frac{1}{2}(y - f(x))^2 & \text{for } |y - f(x)| \leq \delta \\ \delta(|y - f(x)| - \frac{1}{2}\delta) & \text{otherwise} \end{cases}$$

Gradient Descent

(Batch) Gradient Descent

$$\begin{aligned} \min f(a + \Delta x) &\approx \min f(a) + \nabla f(a)^T \Delta x \\ \Rightarrow \nabla f(a)^T \Delta x &\text{ should be as minus as possible} \end{aligned}$$

$$\begin{aligned} v^T w &= \|v\| \|w\| \cos \theta \\ \Rightarrow \nabla f(a)^T \Delta x &= \|\nabla f(a)\| \|\Delta x\| \cos \theta \\ \text{To max minus} &\Rightarrow \cos \theta = -1 \Rightarrow \theta = 180^\circ \\ \Rightarrow \Delta x &= -\alpha \nabla f(a) \end{aligned}$$

$\Rightarrow$

We control the step size with α (learning rate) to minimize $f(a - \alpha \nabla f(a))$.

(Another way) Start from an arbitrary initial weight vector $\vec{w}$.

$$\vec{w}' \leftarrow \vec{w} - \alpha \Delta J(\vec{w}) = \vec{w} - \alpha \frac{\partial}{\partial \vec{w}} J(\vec{w})$$

> α is called learning rate

And for a single training example $(\vec{x}, y)$, we have:

$$\vec{w}' \leftarrow \vec{w} - \alpha(\vec{w} \cdot \vec{x} - y)\vec{x} = \vec{w} + \alpha(y - \vec{w} \cdot \vec{x})\vec{x}$$

Aka. the least mean square (LMS) update rule.

> vs perceptron: $\vec{w}' = \vec{w} + y \cdot \vec{x}$

$$\begin{aligned}
\text{Derivation : } \frac{\partial}{\partial \vec{w}} J(\vec{w}) &= \frac{\partial}{\partial \vec{w}} \frac{1}{2} (f(\vec{x}) - y)^2 \\
&= 2 \cdot \frac{1}{2} (f(\vec{x}) - y) \cdot \frac{\partial}{\partial \vec{w}} (f(\vec{x}) - y) \quad // \text{ Chain Rule} \\
&= (\vec{w} \cdot \vec{x} - y) \cdot \frac{\partial}{\partial \vec{w}} (\vec{w} \cdot \vec{x} - y) \\
&= (\vec{w} \cdot \vec{x} - y) \vec{x}
\end{aligned}$$

Therefore, for the whole dataset D :

$$\vec{w}' \leftarrow \vec{w} + \alpha \sum_{(\vec{x}, y) \in D} (y - \vec{w} \cdot \vec{x}) \vec{x}$$

Stochastic Gradient Descent

Streamline BGD:

$$\vec{w}' \leftarrow \vec{w} + \alpha (y_i - \vec{w} \cdot \vec{x}_i) \vec{x}_i$$

Where $(\vec{x}_i, y_i)$ is a randomly picked training example from D .

Mini-Batch Gradient Descent

> [Balance between GD and SGD](#)

$$\vec{w}' \leftarrow \vec{w} + \alpha \sum_{(\vec{x}, y) \in D_m} (y - \vec{w} \cdot \vec{x}) \vec{x}$$

Where $D_m \subset D$ is a randomly picked m examples of training data.

Stochastic Gradient Descent with Momentum

$$\begin{aligned}
m^{(t+1)} &= \mu m^{(t)} - \alpha \nabla L(\theta^{(t)}) \\
\theta^{(t+1)} &= \theta^{(t)} + m^{(t+1)}
\end{aligned}$$

> E.g. if μ is 0.9, it means we keep 90% old velocity.

> Typically set μ to 0.8 - 0.9 (rule-of-thumb)

Ridge regression gradient descent

> L_2 , because it is differentiable

$$\vec{w}' \leftarrow \vec{w} + \alpha \sum_{(\vec{x}, y) \in D} (y - \vec{w} \cdot \vec{x}) \vec{x} - \lambda \vec{w}$$

The function of $\lambda \vec{w}$ is to decrease the magnitude of $\vec{w}$, leading to reduced model complexity and overfitting.

Normal Equation

TODO

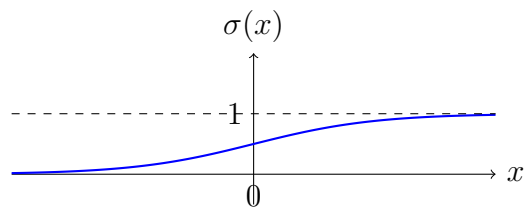
Deep Learning

Activation Functions

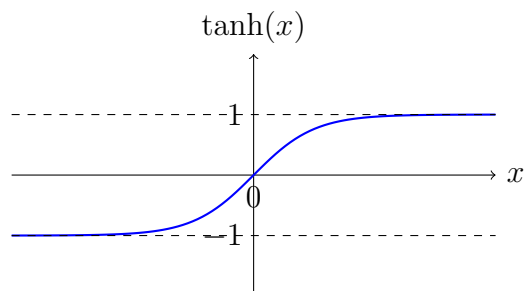
The activation function introduces non-linearity into the model (network), enabling it to learn and represent complex patterns in the data.

> [elementwise](#)

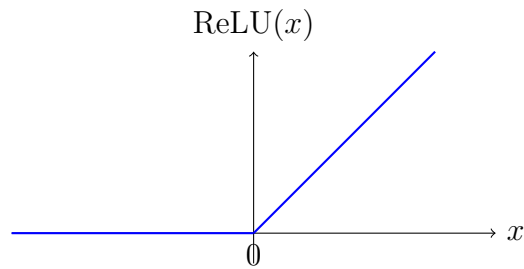
- Sign: $\text{sign}(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases} \Rightarrow$ Simple binary output. (Not differentiable, not good for backpropagation)
- Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}} \Rightarrow$ Usually for binary classification questions.



- Hyperbolic Tangent (Tanh): $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \Rightarrow$ Make data zero-centered which can speed up convergence.



- Rectified Linear Unit (ReLU): $\text{ReLU}(x) = \max(0, x) \Rightarrow$ Only allows positive values to pass through.



> It makes training deep neural networks much easier.

- Leaky ReLU: $\text{Leaky ReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases} \Rightarrow$ Prevents dying ReLU problem.

$$\text{Leaky ReLU}'(x) = \begin{cases} 1 & \text{if } x > 0 \\ \alpha & \text{if } x \leq 0 \end{cases}$$

- Softmax: $\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}} \Rightarrow$ Converts logits into probabilities for multi-class classification. (the only non-elementwise)

Backpropagation

An algorithm way to efficiently compute parameter gradients. $\Rightarrow$ Its work is only until getting $\nabla_{\theta} \text{Loss}$

> Optimizer will use the list generated by backpropagation to update parameters.

For the l-th layer, with weights $W^{(l)}$, the gradient for that layer is:

$$\nabla_{W^{(l)}} L = \delta^{(l)} (\mathbf{a}^{(l-1)})^T$$

$$\mathbf{a}^{(l-1)} \xrightarrow[\text{weights computation}]{W^{(l)}} \mathbf{z}^{(l)} \xrightarrow[\text{activation function}]{\sigma(\cdot)} \mathbf{a}^{(l)}$$

$$> \delta^{(l)} = \frac{\partial L}{\partial \mathbf{z}^{(l)}} = \frac{\partial a^{(l)}}{\partial z^{(l)}} \frac{\partial L}{\partial a^{(l)}} = \sigma'(z^{(l)}) \frac{\partial L}{\partial a^{(l)}}$$

Why and When to use CNN?

- A neuron does not have to see the whole image to discover the pattern.
- The same patterns appear in different locations of the image. $\Rightarrow$ two neurons in different locations can share the same weights.
- Subsampling (pooling) can reduce the dimensionality of the image while preserving important features.

N filters is equivalent to one fully connected layer with a neuron.

The parameters in the filter should be learned like normal weights and biases in NN.

Weight Decay

> Actually, it is almost the synonym of L2 Regularization; it is just from the perspective of update rule.

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla L(\theta^{(t)}) - 2\alpha\lambda\theta$$

Word Embeddings

Cosine Similarity

Cosine similarity measures the relationship between word vectors in an embedding space.

$$\text{Cosine Similarity}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$
$$-1 \geq \text{Cosine Similarity}(\mathbf{A}, \mathbf{B}) \leq 1$$

- 1: Vectors are identical (high semantic similarity)
- 0: Vectors are orthogonal (no semantic similarity)
- -1: ?????

Dimensionality Reduction

E.g. 7D to 2D

> Using e.g. PCA, t-SNE, UMAP

The objective function of the Skip-gram Model

AlphaGo Zero

The Deep Neural Network f_θ

$$(\vec{p}, v) = f_\theta(s)$$

- s : Input, the position and history of the game.
- $\vec{p}$: Output, the probability of selecting each move a (including pass), $p_a = Pr(a \mid s)$.
- v : Output, scalar evaluation, estimating the probability of the current player winning from position s .

MCTS search

In each position s , an MCTS search is executed, guided by the neural network f_θ .

Output π : probabilities of playing each move, usually better than $\vec{p} \Rightarrow$ powerful policy improvement operator.

The idea

Update θ to let $(\vec{p}, v)$ be closer to (π, z) .

> z : the game winner z

Loss function

$$l = (z - v)^2 - \pi^T \log \vec{p} + c \|\theta\|^2$$

The neural network parameters θ are updated to **maximize** the similarity of the policy vector $\vec{p}$ to the search probabilities π_t , and to **minimize** the error between the predicted winner v_t and the game winner z .